

Arquitectura de Ecosistemas Digitales: Implementación de Server-Driven UI con Material Design 3 y Automatización mediante Figma MCP

1. Resumen Ejecutivo y Visión Estratégica

La industria del desarrollo móvil se encuentra en un punto de inflexión crítico. El modelo tradicional de distribución de aplicaciones, caracterizado por ciclos de lanzamiento rígidos, dependencia de las revisiones de las tiendas de aplicaciones (Google Play Store y Apple App Store) y una disonancia operativa entre los equipos de diseño y desarrollo, está siendo reemplazado por arquitecturas más dinámicas y reactivas. Este informe profundiza en la investigación y el diseño de una arquitectura de **Server-Driven UI (SDUI)** para Android, utilizando **Jetpack Compose** y **Material Design 3 (MD3)**, orquestada por un Dashboard de gestión inteligente e impulsada por la reciente innovación del **Model Context Protocol (MCP)** de Figma.

El objetivo central de esta propuesta es establecer un ecosistema donde la interfaz de usuario no sea un artefacto estático compilado en el binario de la aplicación, sino un recurso dinámico servido bajo demanda. Esto permite una agilidad de negocio sin precedentes: la capacidad de alterar flujos de conversión, realizar pruebas A/B complejas y desplegar nuevas funcionalidades visuales sin requerir una actualización de la aplicación en el dispositivo del usuario.¹

Sin embargo, la implementación de SDUI conlleva una complejidad inherente en la gestión de esquemas de datos y la sincronización entre el diseño visual y la estructura JSON que lo alimenta. Históricamente, este paso requería una traducción manual propensa a errores. La integración del **Figma MCP**, un estándar abierto que permite a los agentes de Inteligencia Artificial leer semánticamente la estructura de diseño, cierra esta brecha, permitiendo un flujo de trabajo "Diseño a Código" casi automatizado.³

A continuación, se presenta un análisis exhaustivo que abarca desde la teoría arquitectónica hasta los detalles de implementación en Kotlin y la configuración de infraestructura de backend.

2. Paradigma Arquitectónico: Server-Driven UI (SDUI)

2.1. Deconstrucción del Modelo Tradicional vs. SDUI

En el desarrollo nativo convencional, la lógica de presentación y la lógica de negocio a menudo se entrelazan en el cliente. El servidor actúa meramente como un proveedor de datos crudos (entidades de dominio como Producto, Usuario), y es responsabilidad del cliente decidir cómo renderizar esos datos. Este acoplamiento fuerte significa que cualquier cambio en la jerarquía visual requiere modificar el código fuente, recompilar y redistribuir el binario.¹

El Server-Driven UI invierte esta responsabilidad. El servidor no envía datos de dominio, sino **instrucciones de renderizado**. Envía un árbol de componentes que describe *qué* debe mostrarse (un botón, una imagen, una fila) y *cómo* debe comportarse. El cliente Android se convierte, efectivamente, en un motor de renderizado de componentes nativos, agnóstico al dominio del negocio.⁶

Tabla 2.1: Comparativa de Responsabilidades Arquitectónicas

Característica	Desarrollo Nativo Tradicional	Server-Driven UI (SDUI)
Fuente de la Verdad UI	Código Cliente (Kotlin/XML)	Respuesta del Servidor (JSON)
Ciclo de Actualización	Semanas (App Store Review)	Inmediato (Siguiente Request)
Lógica de Presentación	Hardcodeada en el Cliente	Definida en el Backend
Consistencia	Fragmentada por versión de App	Unificada para todos los usuarios
Carga de Red	Baja (Solo datos puros)	Media/Alta (Estructura + Datos)
Complejidad de Mant.	Lineal	Exponencial en el esquema JSON

2.2. Enfoque Híbrido: La Estrategia Recomendada

Aunque la teoría sugiere un SDUI puro, la evidencia empírica de implementaciones a gran escala como las de Airbnb y Lyft sugiere que un **Enfoque Híbrido** es superior para garantizar la calidad de la experiencia del usuario (UX).⁶ Las pantallas que requieren interacciones

complejas, acceso a hardware de baja latencia (cámara, sensores) o animaciones gestuales intensivas deben permanecer nativas. El SDUI brilla en áreas de alto cambio y contenido dinámico: el *Feed* principal, las páginas de aterrizaje de marketing, el *Checkout* y los flujos de *Onboarding*.⁶

En este modelo, el cliente Android mantiene un "Registro de Componentes" (Component Registry). El servidor envía un JSON que referencia las claves de este registro. Si el servidor solicita un componente que el cliente no tiene (por ejemplo, en una versión antigua de la app), el cliente debe tener mecanismos de degradación elegante (fallback), mostrando un espacio vacío o un componente genérico compatible, evitando así cierres inesperados.¹⁰

2.3. Definición del Contrato de Datos (JSON Schema)

El éxito del SDUI depende de la robustez de su esquema JSON. No se trata simplemente de serializar vistas, sino de definir un lenguaje de dominio específico (DSL) en formato JSON. El análisis de esquemas utilizados por Airbnb (Ghost Platform) y otras soluciones propietarias revela una estructura tripartita esencial: Estructura, Datos y Comportamiento.⁸

1. **Tipo de Componente (type):** El identificador único que mapea con el registro del cliente (e.g., carousel, hero_card, action_button).
2. **Propiedades de Datos (data):** El contenido que puebla el componente. No debe contener lógica, solo valores finales o referencias a recursos.
3. **Modificadores de Estilo (style):** Aquí es donde Material Design 3 juega un papel crucial. En lugar de enviar valores hexadecimales (#FF0000) o dimensiones en píxeles (16px), se envían **Tokens de Diseño** (sys.color.primary, sys.spacing.medium). Esto permite que la aplicación resuelva el estilo según el tema actual (Claro/Oscuro) y la configuración del dispositivo, manteniendo la fidelidad nativa.¹²
4. **Acciones y Comportamiento (actions):** Definición de lo que sucede al interactuar. Puede ser una navegación (deep_link), una mutación de estado local o una llamada a la API.

3. Sistema de Diseño: Profundización en Material Design 3

La adopción de Material Design 3 (MD3) no es meramente estética; proporciona un sistema semántico estandarizado que es vital para la abstracción requerida en SDUI. En un entorno donde el servidor desconoce el contexto del dispositivo (si el usuario tiene activado el modo oscuro o el contraste alto), MD3 actúa como el traductor universal.¹⁴

3.1. Taxonomía de Tokens de Diseño

La arquitectura de tokens es la columna vertebral de la consistencia visual en un sistema

distribuido. Se propone una estructura jerárquica de tres niveles, alineada con las mejores prácticas de Figma y el estándar de Google.¹⁶

3.1.1. Tokens de Referencia (Primitivos)

Representan los valores base del sistema, derivados directamente de la marca, pero sin contexto de uso.

- *Ejemplo:* `ref.palette.blue.400 = #42A5F5`
- *Uso:* **Prohibido** en el JSON de SDUI. Su uso directo rompe la adaptabilidad temática.

3.1.2. Tokens de Sistema (Semánticos)

Describen la intención del diseño. Son la interfaz pública del sistema de diseño hacia el SDUI.

- *Ejemplo:* `sys.color.primary.container`, `sys.typography.headlineLarge`, `sys.spacing.layout.page_margin`.
- *Uso:* **Obligatorio** en el JSON. Al usar `sys.color.background`, el cliente Android sabe automáticamente aplicar `#FFFFFF` en modo claro y `#121212` en modo oscuro, sin que el servidor intervenga. Esto reduce drásticamente el tamaño del payload JSON, ya que no es necesario enviar variantes de estilo para cada modo.¹⁷

3.1.3. Tokens de Componente

Mapean tokens de sistema a propiedades específicas de un widget.

- *Ejemplo:* `comp.card.outlined.stroke.color` -> `sys.color.outline`.
- *Uso:* Generalmente resueltos internamente por la librería de componentes en Android, aunque pueden exponerse en el JSON para permitir "overrides" específicos (ej. una tarjeta promocional con un borde especial).

3.2. Exportación y Sincronización desde Figma

Para que el sistema funcione, la definición de tokens en Figma debe ser idéntica a la implementación en el código Android. El uso de plugins como **Material Theme Builder** o **Tokens Studio** permite exportar estas definiciones a archivos JSON (ej. `theme.json`). Este archivo maestro debe ser consumido tanto por el Dashboard de gestión (para previsualización) como por la aplicación Android (durante la compilación o dinámicamente al inicio) para asegurar que ambos sistemas "hablen el mismo idioma" visual.¹⁶

4. La Revolución del Model Context Protocol (MCP)

La mayor barrera de entrada para SDUI ha sido la gestión del contenido. Crear los archivos JSON manualmente es inviable a escala, y desarrollar editores visuales "Drag-and-Drop" (tipo Wix o Squarespace) es extremadamente costoso. Aquí es donde el **Model Context Protocol**

(MCP) de Figma introduce un cambio de paradigma radical.³

4.1. Arquitectura del MCP en Figma

El MCP es un estándar abierto desarrollado por Anthropic que permite conectar Modelos de Lenguaje (LLMs) con fuentes de datos externas y herramientas. En el contexto de Figma, el **Servidor MCP de Figma** expone la estructura interna del archivo de diseño a agentes de IA (como Claude Desktop, Cursor o Windsurf).²¹

A diferencia de las herramientas anteriores que intentaban "ver" el diseño mediante capturas de pantalla (Computer Vision), el MCP accede al **Árbol de Nodos (DOM)** de Figma. El agente recibe información estructurada y precisa:

- Identifica que un rectángulo es un Frame con AutoLayout: Vertical.
- Lee que el relleno es una variable vinculada a `sys.color.surface`.
- Reconoce instancias de componentes, variantes y propiedades de texto.

4.2. Flujo de Trabajo Automatizado: Diseño a JSON

La propuesta de valor central de este blueprint es la utilización de MCP para automatizar la generación del esquema SDUI. El flujo operativo recomendado es el siguiente:

1. **Conexión:** El desarrollador o el sistema de CI/CD se conecta al Servidor MCP de Figma (local o remoto).²³
2. **Selección y Contexto:** Se selecciona un Frame en Figma que representa una pantalla o sección de la app.
3. **Ingeniería de Prompts (System Prompt):** Se instruye al agente de IA con un "System Prompt" riguroso que contiene la definición del esquema JSON del SDUI.
 - *Ejemplo de Instrucción:* "Actúa como un motor de exportación SDUI. Analiza el nodo seleccionado. Transforma los Frames con AutoLayout vertical en componentes type: 'column'. Mapea todas las variables de color a sus nombres de token (ej. `sys.color.primary`). Si encuentras una instancia de componente llamada 'ProductCard', genera un objeto JSON type: 'product_card' y extrae sus propiedades de texto como datos. La salida debe ser estrictamente JSON válido."²⁵
4. **Generación:** El agente procesa la estructura de Figma y genera el bloque JSON listo para ser insertado en el Dashboard.²²

Este flujo elimina el error humano en la transcripción de estilos y garantiza que la estructura del JSON refleje fielmente la jerarquía visual diseñada, incluyendo los tokens correctos de Material Design 3.²⁷

5. Arquitectura del Backend y Dashboard de Gestión

Para orquestar este ecosistema, se requiere una plataforma centralizada que almacene las

configuraciones JSON, gestione versiones y sirva las respuestas a la aplicación móvil. Existen dos caminos divergentes: utilizar un Headless CMS existente o construir una solución a medida.

5.1. Análisis Crítico: Headless CMS (Strapi)

Strapi es un candidato fuerte debido a su naturaleza de código abierto y extensibilidad. Sus "Dynamic Zones" (Zonas Dinámicas) parecen, en teoría, la solución perfecta para SDUI, permitiendo mezclar componentes libremente.

5.1.1. El Problema del Anidamiento en Strapi v5

Una investigación profunda revela una limitación crítica en las versiones recientes de Strapi (v5). Para evitar problemas de rendimiento en la base de datos y bucles infinitos en la API, Strapi ha limitado la capacidad de anidar componentes dentro de Zonas Dinámicas a una profundidad muy baja (típicamente 2 niveles en la UI).²⁹

Esto es fatal para SDUI, donde una estructura típica podría ser: Pantalla > Scroll Vertical > Sección > Carrusel > Tarjeta > Fila > Texto. Esta jerarquía tiene 6 o 7 niveles de profundidad.

5.1.2. Soluciones Arquitectónicas para Strapi

Si se opta por Strapi para el MVP, se deben implementar "workarounds" arquitectónicos:

1. **Relaciones Recursivas (Tree Structure):** En lugar de usar la UI de anidamiento visual, se modelan los componentes como entidades independientes que tienen relaciones "HasMany" consigo mismas (campo children). Esto permite una profundidad infinita, pero la experiencia de edición se vuelve fragmentada (el editor debe crear los "hijos" por separado y luego vincularlos).³²
2. **Campo JSON Crudo (Raw JSON):** La solución más pragmática para un MVP impulsado por MCP. Se define un único campo de tipo JSON en el Content Type "Page". El JSON completo, generado por Figma MCP, se pega directamente en este campo.
 - *Ventaja:* Flexibilidad total, sin límites de anidamiento.
 - *Desventaja:* Se pierde la validación visual de Strapi.
 - *Mitigación:* Implementar un **Plugin de Validación** personalizado en Strapi que use una librería de JSON Schema Validator para rechazar guardados si el JSON no cumple con la estructura requerida.³³

5.2. Alternativa: Dashboard Personalizado (Next.js + MCP)

Para un producto a largo plazo, un Dashboard a medida construido con Next.js ofrece la mejor experiencia.

- **Integración Directa con MCP:** El Dashboard puede tener un botón "Importar desde Figma". Al pulsarlo, el backend del Dashboard se conecta a la API de Figma (o actúa como cliente MCP), lee el diseño y pre-llena el JSON, eliminando el "Copy-Paste"

manual.³⁵

- **Previsualización Web:** Al compartir la misma lógica de renderizado (o una aproximación en React) que la app móvil, los editores pueden ver cómo quedará la pantalla antes de publicar.
- **Gestión de Versiones:** Permite implementar lógica compleja de versionado, sirviendo diferentes estructuras JSON según la versión de la app del usuario (User-Agent), gestionando la compatibilidad hacia atrás.³⁶

6. Implementación Técnica del Cliente Android (MVP)

El cliente Android debe ser robusto, eficiente y tolerante a fallos. La implementación se basa en Jetpack Compose y Kotlin.

6.1. Serialización Polimórfica con `kotlinx.serialization`

El desafío técnico principal es deserializar una lista heterogénea de objetos. El JSON contiene un array children donde el primer elemento puede ser un Text y el segundo un Image.

Utilizar `kotlinx.serialization` es superior a GSON o Moshi en este contexto debido a su soporte nativo para polimorfismo y su eficiencia en tiempo de ejecución (sin reflexión costosa).³⁷

Implementación del Modelo Base:

Kotlin

```
@Serializable
sealed interface UiComponent {
    val modifier: UiModifier?
}
```

```
@Serializable
@SerialName("column") // Discriminador para el JSON "type": "column"
data class ColumnComponent(
    val children: List<UiComponent>,
    override val modifier: UiModifier? = null
) : UiComponent
```

```
@Serializable
@SerialName("button")
data class ButtonComponent(
```

```

    val text: String,
    val action: UiAction,
    override val modifier: UiModifier? = null
) : UiComponent

```

El serializador inspecciona automáticamente el campo type (configuración por defecto para discriminadores polimórficos) y decide qué clase instanciar.

6.2. Registro de Componentes (Component Registry)

Para transformar estos objetos de datos en funciones @Composable, se debe evitar el uso de reflexión (Class.forName) para mapear cadenas a funciones, ya que R8/Proguard ofusca los nombres de clases en producción y la reflexión es lenta.³⁹

El patrón recomendado es un **Registro Estático** o una función de despacho central:

Kotlin

```

@Composable
fun RenderUiComponent(component: UiComponent) {
    when (component) {
        is ColumnComponent -> ServerColumn(component)
        is ButtonComponent -> ServerButton(component)
        is TextComponent -> ServerText(component)
        // El 'else' es vital para la compatibilidad futura
        else -> FallbackComponent(component)
    }
}

```

6.3. Integración de Lógica Dinámica (JSON Logic)

Para que el SDUI sea verdaderamente poderoso, necesita soportar lógica condicional simple (ej. "Mostrar este banner solo si el usuario no está logueado"). En lugar de inventar un lenguaje propio, se recomienda integrar **JsonLogic**, un estándar para escribir reglas lógicas en JSON.⁴¹

Se integra una librería de evaluación JsonLogic en Kotlin (como json-logic-kotlin).⁴² El componente JSON incluye una propiedad visibility_rule:

JSON

```
"visibility_rule": {  
  "and": [  
    { "==": [{ "var": "user.is_premium" }, false] },  
    { ">": [{ "var": "app_open_count" }, 5] }  
  ]  
}
```

Antes de renderizar, el cliente evalúa esta regla contra el contexto local. Si el resultado es false, el componente se omite del árbol de composición.

7. Blueprint: Plan de Análisis, Diseño e Implementación

A continuación, se detalla el plan de ejecución faseado para desarrollar el Dashboard y el MVP.

Fase 1: Fundamentos y Definición (Semanas 1-2)

- **Objetivo:** Establecer el contrato de datos y el lenguaje visual.
- **Actividades:**
 1. **Configuración de Figma:**
 - Crear librería de estilos MD3.
 - Definir nombres semánticos para tokens (sys.color.*).
 - Instalar plugin "Material Theme Builder" y exportar tokens.json.
 2. **Definición del Schema JSON:**
 - Documentar los campos type, data, style, action.
 - Definir la lista inicial de componentes soportados (MVP: Column, Row, Text, Image, Button, Spacer).
 3. **Configuración del Entorno MCP:**
 - Instalar Cursor o configurar Claude Desktop con el servidor MCP de Figma.
 - Desarrollar y probar el "System Prompt" para asegurar que la IA genere JSON que cumpla estrictamente con el Schema definido.²⁶

Fase 2: Infraestructura de Backend (Semanas 3-4)

- **Objetivo:** Crear el repositorio de UI y la API.
- **Actividades:**
 1. **Despliegue de Strapi (MVP):**
 - Crear Content Type Screen con campos: slug (UID), description (Text), schema_version (Int), layout_config (JSON Field).

- Implementar middleware de validación para asegurar que el JSON en `layout_config` es válido.
2. **API:**
- Configurar endpoints REST: GET `/api/screens/{slug}`.
 - Implementar caché (Redis o CDN) para asegurar latencia mínima en la entrega de JSON.⁶

Fase 3: Desarrollo del Cliente Android (Semanas 4-6)

- **Objetivo:** Construir el motor de renderizado SDUI.
- **Actividades:**
 1. **Capa de Datos:** Implementar clases selladas `UiComponent` y configuración de `JsonContentPolymorphicSerializer` en Kotlin.³⁷
 2. **Sistema de Diseño:** Crear el `MaterialTheme` dinámico que lea los tokens. Implementar funciones de utilidad para mapear tokens de string ("`sys.color.primary`") a instancias de color de Compose.
 3. **Renderizado Recursivo:** Implementar la función `RenderUiComponent` y los composables atómicos (`ServerText`, `ServerButton`).
 4. **Navegación:** Implementar un `NavigationHandler` que interprete acciones de Deep Link (`app://product/123`) desde el JSON.⁴⁴

Fase 4: Integración y Lanzamiento (Semanas 7-8)

- **Objetivo:** Conectar las piezas y validar el flujo.
- **Actividades:**
 1. **Prueba de Concepto (End-to-End):**
 - Diseñar pantalla "Home" en Figma.
 - Usar Prompt MCP para extraer JSON.
 - Pegar JSON en Strapi.
 - Verificar renderizado en App Android.
 2. **Optimización:**
 - Implementar persistencia local (Room o DataStore) para guardar el último JSON conocido y mostrarlo offline (estrategia *stale-while-revalidate*).⁶
 - Medir tiempos de renderizado y optimizar listas largas (`LazyColumn`) en el renderizador SDUI.

8. Análisis de Riesgos y Estrategias de Mitigación

8.1. Desincronización de Versiones

Riesgo: El servidor envía un componente `VideoPlayer` que la versión instalada de la app (v1.0) no conoce, causando un crash.

Mitigación:

- Implementar un bloque else (fallback) en el registro de componentes que renderice un Box vacío de tamaño 0 o un placeholder "Actualiza tu app".
- Incluir min_app_version en la respuesta JSON. Si la app es muy antigua, forzar una pantalla nativa de "Actualización Requerida".¹⁰

8.2. Rendimiento de la Red

Riesgo: Pantallas en blanco mientras se descarga el JSON en conexiones lentas.

Mitigación:

- **Prefetching:** Descargar los JSONs de las pantallas más probables (ej. detalle de producto) en segundo plano antes de que el usuario navegue a ellas.
- **Bundling:** Incluir una versión base del JSON en los assets de la aplicación para el primer arranque.⁶

8.3. Alucinaciones de la IA (MCP)

Riesgo: El agente MCP genera un JSON sintácticamente correcto pero con tokens de color inventados o estructura lógica errónea.

Mitigación:

- Validación estricta con **JSON Schema** en el Dashboard antes de guardar.
- El "System Prompt" debe incluir una lista cerrada de tokens permitidos y componentes válidos para restringir la "creatividad" del modelo.²⁶

9. Conclusión

La implementación de una arquitectura Server-Driven UI potenciada por Material Design 3 y Figma MCP no es solo una mejora tecnológica, sino una transformación operativa. Al tratar la interfaz de usuario como datos gestionados, se desbloquea la capacidad de iterar a la velocidad del pensamiento de diseño, eliminando las barreras de distribución tradicionales.

El uso estratégico de **Figma MCP** actúa como el catalizador que hace viable esta arquitectura, reduciendo drásticamente el coste de mantenimiento del esquema JSON. Aunque desafíos como la gestión de versiones y el anidamiento en el CMS requieren soluciones arquitectónicas cuidadosas (como el uso de JSON crudo o Dashboards personalizados), los beneficios en agilidad y consistencia multiplataforma justifican plenamente la inversión. Este blueprint proporciona una base sólida, escalable y moderna para construir la próxima generación de experiencias móviles en Android.

Fuentes citadas

1. The Server-Driven UI Dilemma: A Pragmatic Guide for the Modern Mobile Developer, acceso: enero 4, 2026, <https://pankaj-rai.medium.com/the-server-driven-ui-dilemma-a-pragmatic-guide-for-the-modern-mobile-developer-b45b80d0bff3>

2. Server Driven UI - DEV Community, acceso: enero 4, 2026,
https://dev.to/nishant_keshav/server-driven-ui-3l0p
3. How to structure Figma files for MCP and AI-powered code generation - LogRocket Blog, acceso: enero 4, 2026,
<https://blog.logrocket.com/ux-design/design-to-code-with-figma-mcp/>
4. Figma MCP: Complete Guide to Design-to-Code Automation - Seamgen, acceso: enero 4, 2026,
<https://www.seamgen.com/blog/figma-mcp-complete-guide-to-design-to-code-automation>
5. Server-Driven UI for Android with Jetpack Compose | Bottle Rocket - News & Views, acceso: enero 4, 2026,
<https://blog.bottlerocketstudios.com/server-driven-ui-for-android-with-jetpack-compose>
6. RemoteCompose: Another Paradigm for Server-Driven UI in Jetpack Compose | by Jaewoong Eum | Nov, 2025 | ProAndroidDev, acceso: enero 4, 2026,
<https://proandroiddev.com/remotecompose-another-paradigm-for-server-driven-ui-in-jetpack-compose-92186619ba8f>
7. Server-Driven UI Design Patterns: A Professional Guide with Examples - Dev Cookies, acceso: enero 4, 2026,
<https://devcookies.medium.com/server-driven-ui-design-patterns-a-professional-guide-with-examples-a536c8f9965f>
8. A Deep Dive into Airbnb's Server-Driven UI System | by Ryan Brooks - Medium, acceso: enero 4, 2026,
<https://medium.com/airbnb-engineering/a-deep-dive-into-airbnbs-server-driven-ui-system-842244c5f5>
9. Design Server-Driven UI with Jetpack Compose and Firebase - GetStream.io, acceso: enero 4, 2026,
<https://getstream.io/blog/server-driven-compose-firebase/>
10. Server-driven UI (or Backend driven UI) strategies · MobileNativeFoundation discussions · Discussion #47 - GitHub, acceso: enero 4, 2026,
<https://github.com/MobileNativeFoundation/discussions/discussions/47>
11. Airbnb's Server-Driven UI System - Hacker News, acceso: enero 4, 2026,
<https://news.ycombinator.com/item?id=27707423>
12. Server-Driven UI: Agile Interfaces Without App Releases - DZone, acceso: enero 4, 2026,
<https://dzone.com/articles/server-driven-ui-agile-interfaces-without-app-releases>
13. Design tokens – Material Design 3, acceso: enero 4, 2026,
<https://m3.material.io/foundations/design-tokens>
14. Jetpack Compose UI App Development Toolkit - Android Developers, acceso: enero 4, 2026, <https://developer.android.com/compose>
15. Material Design 3 in Compose | Jetpack Compose - Android Developers, acceso: enero 4, 2026,
<https://developer.android.com/develop/ui/compose/designsystems/material3>
16. Design tokens – Material Design 3, acceso: enero 4, 2026,

- <https://m3.material.io/foundations/design-tokens/how-to-use-tokens>
17. Exporting Figma Variables to Code: A Step-by-Step Workflow - Design Systems Collective, acceso: enero 4, 2026,
<https://www.designsystemscollective.com/exporting-figma-variables-to-code-a-step-by-step-workflow-3f793def100b>
 18. Customizing Material – Material Design 3, acceso: enero 4, 2026,
<https://m3.material.io/foundations/customization>
 19. Export Figma Variables or Design Tokens to JSON - The Design System Guide, acceso: enero 4, 2026,
<https://thedesignteam.guide/knowledge-base/export-figma-variables-or-design-tokens-to-json>
 20. Introducing Material Theme Builder, acceso: enero 4, 2026,
<https://m3.material.io/blog/material-theme-builder>
 21. Design to Code with the Figma MCP Server - Builder.io, acceso: enero 4, 2026,
<https://www.builder.io/blog/figma-mcp-server>
 22. Guide to the Figma MCP server, acceso: enero 4, 2026,
<https://help.figma.com/hc/en-us/articles/32132100833559-Guide-to-the-Figma-MCP-server>
 23. A guide on how to use the Figma MCP server - GitHub, acceso: enero 4, 2026,
<https://github.com/figma/mcp-server-guide>
 24. Claude Code + Figma MCP Server - Builder.io, acceso: enero 4, 2026,
<https://www.builder.io/blog/claude-code-figma-mcp-server>
 25. How to Turn Your Figma Designs into Real Apps Using MCP and Cursor (Step-by-Step Guide) - Sergei Chyrkov, acceso: enero 4, 2026,
[https://sergeichyrkov.com/blog/how-to-turn-your-figma-designs-into-real-apps-using-mcp-and-cursor-\(step-by-step-guide\)](https://sergeichyrkov.com/blog/how-to-turn-your-figma-designs-into-real-apps-using-mcp-and-cursor-(step-by-step-guide))
 26. Write effective prompts to guide the AI | Developer Docs, acceso: enero 4, 2026,
<https://developers.figma.com/docs/figma-mcp-server/write-effective-prompts/>
 27. The Complete Guide to Figma's MCP Integration: From Design to Code - Garima Dua, acceso: enero 4, 2026,
<https://garimadua.medium.com/the-complete-guide-to-figmas-mcp-integration-from-design-to-code-7e20d792b053>
 28. Bridging Design and Code: A Comprehensive Guide to Figma MCP, Code Connect, and Figma Make | by Michel Velis | Nov, 2025 | Medium, acceso: enero 4, 2026,
<https://medium.com/@mvelis90/bridging-design-and-code-a-comprehensive-guide-to-figma-mcp-code-connect-and-figma-make-1d5c5afef8fe>
 29. How to work around nesting components / dynamic zones : r/Strapi - Reddit, acceso: enero 4, 2026,
https://www.reddit.com/r/Strapi/comments/szwjnk/how_to_work_around_nesting_components_dynamic/
 30. Nested Components (more than 2 levels from the UI) | Content Editing XP - Strapi, acceso: enero 4, 2026,
<https://feedback.strapi.io/customization/p/nested-components-more-than-2-levels-from-the-ui>

31. Allow nesting of components more than 2 levels deep in UI | Voters - Strapi, accesso: enero 4, 2026, <https://feedback.strapi.io/feature-requests/p/allow-nesting-of-components-more-than-2-levels-deep-in-ui>
32. 5 Key SQL Relationship Types to Improve Database Design - Strapi, accesso: enero 4, 2026, <https://strapi.io/blog/5-key-sql-relationship-types-to-improve-database-design>
33. Build a Custom Strapi Plugin with Medium and Dev.to APIs - Part 1, accesso: enero 4, 2026, <https://strapi.io/blog/build-a-custom-strapi-plugin-with-medium-and-devto-apis-part1>
34. Serialize and Deserialize JSON with Strapi - MojoAuth, accesso: enero 4, 2026, <https://mojoauth.com/serialize-and-deserialize/serialize-and-deserialize-json-with-strapi/>
35. Headless CMS vs Custom CMS for B2B Apps: Key Comparison Guide - Metizsoft Solutions, accesso: enero 4, 2026, <https://www.metizsoft.com/blog/headless-cms-vs-custom-cms>
36. skydoves/server-driven-compose - GitHub, accesso: enero 4, 2026, <https://github.com/skydoves/server-driven-compose>
37. JsonContentPolymorphicSerializer | kotlinx.serialization - Kotlin Programming Language, accesso: enero 4, 2026, <https://kotlinlang.org/api/kotlinx.serialization/kotlinx-serialization-json/kotlinx.serialization.json/-json-content-polymorphic-serializer/>
38. Kotlin serialization library for JSON serialization | Lightricks Tech Blog - Medium, accesso: enero 4, 2026, <https://medium.com/lightricks-tech-blog/using-the-kotlin-serialization-library-for-tough-json-serialization-82f8b7ae70dc>
39. Reflection | Kotlin Documentation, accesso: enero 4, 2026, <https://kotlinlang.org/docs/reflection.html>
40. Using Kotlin Reflection to Automate Compose Navigation | by jim - ProAndroidDev, accesso: enero 4, 2026, <https://proandroiddev.com/using-kotlin-reflection-to-automate-compose-navigation-4c5b565f660f>
41. JsonLogic, accesso: enero 4, 2026, <https://jsonlogic.com/>
42. advantagefse/json-logic-kotlin - GitHub, accesso: enero 4, 2026, <https://github.com/advantagefse/json-logic-kotlin>
43. allegro/json-logic-kmp: Kotlin multiplatform JsonLogic expressions evaluation engine. Targets iOS and JVM (also Android). - GitHub, accesso: enero 4, 2026, <https://github.com/allegro/json-logic-kmp>
44. Navigate between screens with Compose - Android Developers, accesso: enero 4, 2026, <https://developer.android.com/codelabs/basic-android-kotlin-compose-navigation>
45. Server Driven UI - is this really worth it? : r/androiddev - Reddit, accesso: enero 4, 2026,

https://www.reddit.com/r/androiddev/comments/1ejngo9/server_driven_ui_is_this_really_worth_it/